Presentation Scientific computing

LINMA2710

D. Guerand

Contents

Part 1 : Basic matrix operations and SIMD

Row-major storage & optimisations

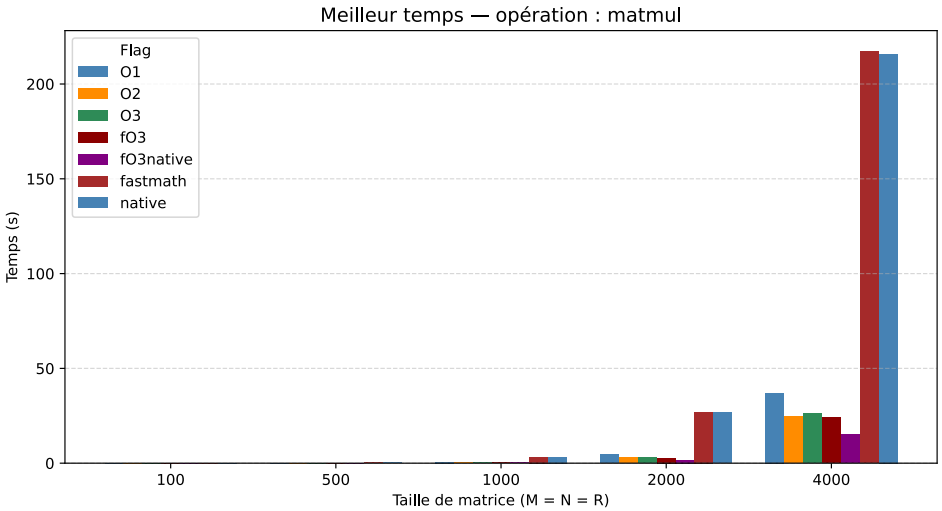
- Matrices stored in **row-major** order → transposing B^T before multiply avoids column-stride cache misses
- **Tilling** splits the computation into small blocks that fit in L1/L2 cache (detailed in Part 2)
- Compiler flags: `O1`, `O2`, `O3 -march=native` enable auto-vectorisation (SIMD)
- Unrolling and helping for vectorisation
- `__restrict__` : Two values do not overlap
- `collapse(2)` : Tells the compiler that the next two loops can be parallelised
- `schedule(static)`

Row-major storage & optimisations

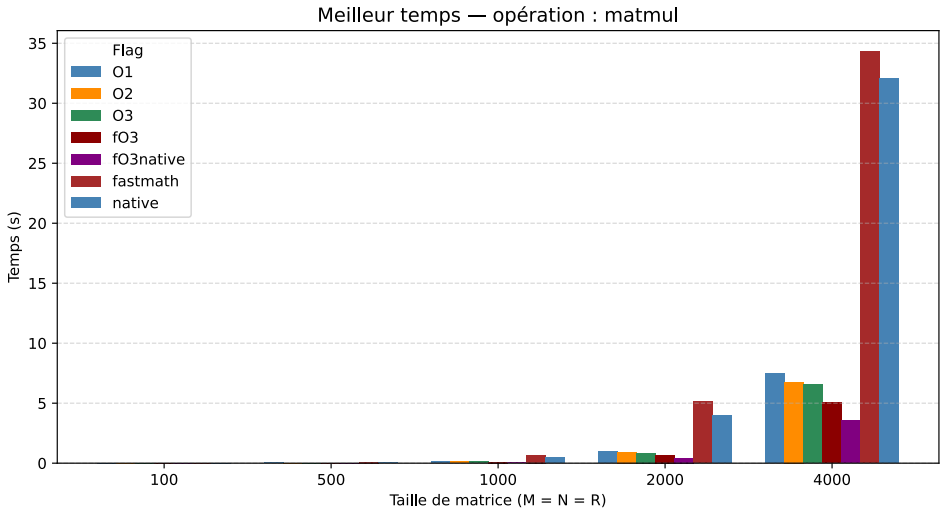

```
for (int j = jj; j < jEnd - 7; j += 8)
                {
                    double sum0 = 0.0, sum1 = 0.0, sum2 = 0.0, sum3 = 0.0, sum4 = 0.0, sum5 =0.0 , sum6=0.0 , sum7 = 0.0;

                    for (int k = kk; k < kEnd; k++)
                    {
                        double a = rowA[k];
                        sum0 += a * BT[(j+0) * K + k];
                        sum1 += a * BT[(j+1) * K + k];
                        sum2 += a * BT[(j+2) * K + k];
                        sum3 += a * BT[(j+3) * K + k];
                        sum4 += a * BT[(j+4) * K + k];
                        sum5 += a * BT[(j+5) * K + k];
                        sum6 += a * BT[(j+6) * K + k];
                        sum7 += a * BT[(j+7) * K + k];

                    }

                    Res[i * C + (j+0)] += sum0;
                    Res[i * C + (j+1)] += sum1;
                    Res[i * C + (j+2)] += sum2;
                    Res[i * C + (j+3)] += sum3;
                    Res[i * C + (j+4)] += sum4;
                    Res[i * C + (j+5)] += sum5;
                    Res[i * C + (j+6)] += sum6;
                    Res[i * C + (j+7)] += sum7;

                }
```

Operation times — matmul

Server (with tiling)



My PC

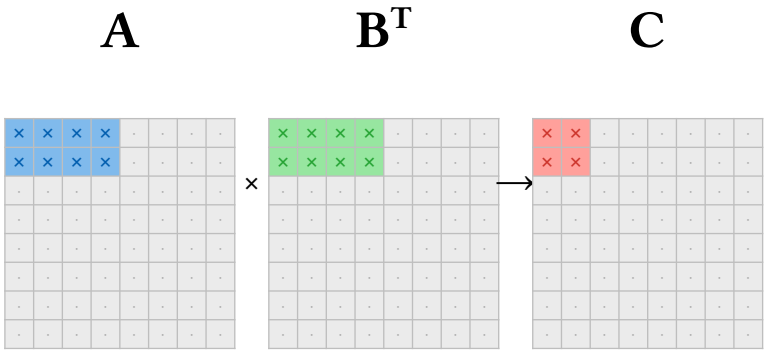


Part 2 : OpenMP

Tiling — principle

- Splits the computation into **blocks (tiles)** that fit in L2 cache
- The tile of **A** stays in cache for the entire `jj` loop
- Read `C/TILE_J` times **from cache**, not from RAM
- ↑ arithmetic intensity ⟶ approche la limite roofline

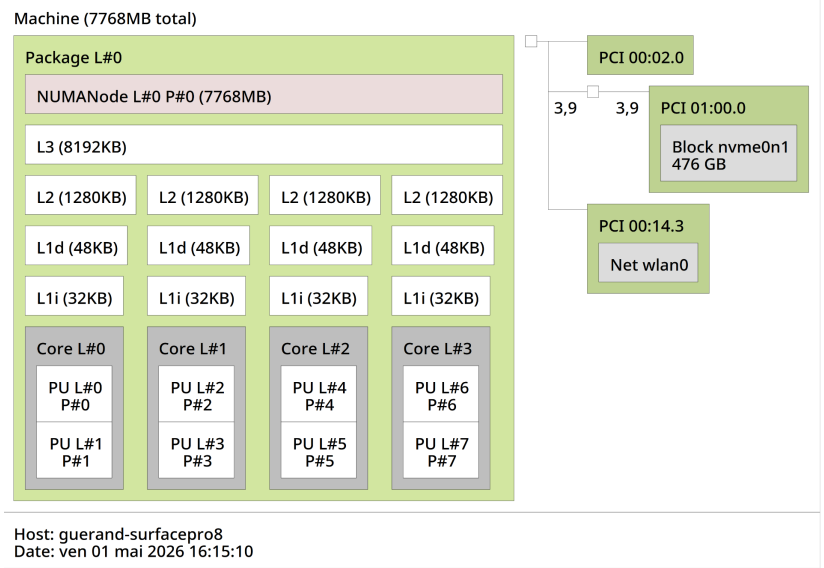
8×8 matrix, proportional to TILE 64/64/32



Tilling — continued

Chosen tile sizes

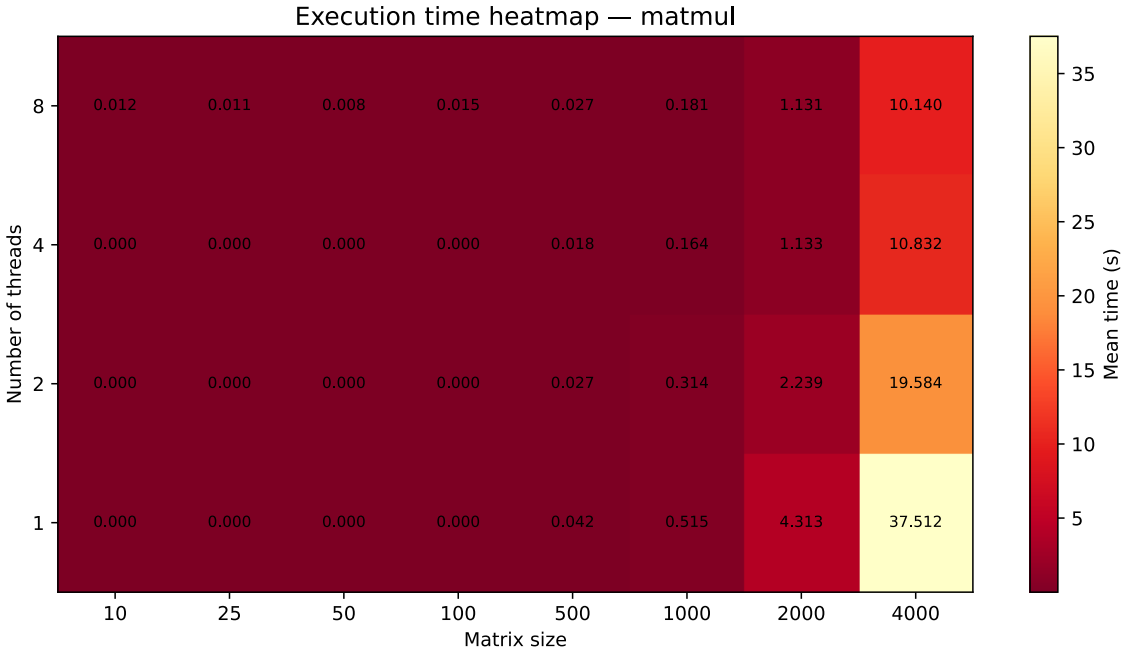
- **My PC:** `TILE_I = TILE_J = TILE_K = 64`
- **Server:** `TILE_I = TILE_J = 64, TILE_K = 32`



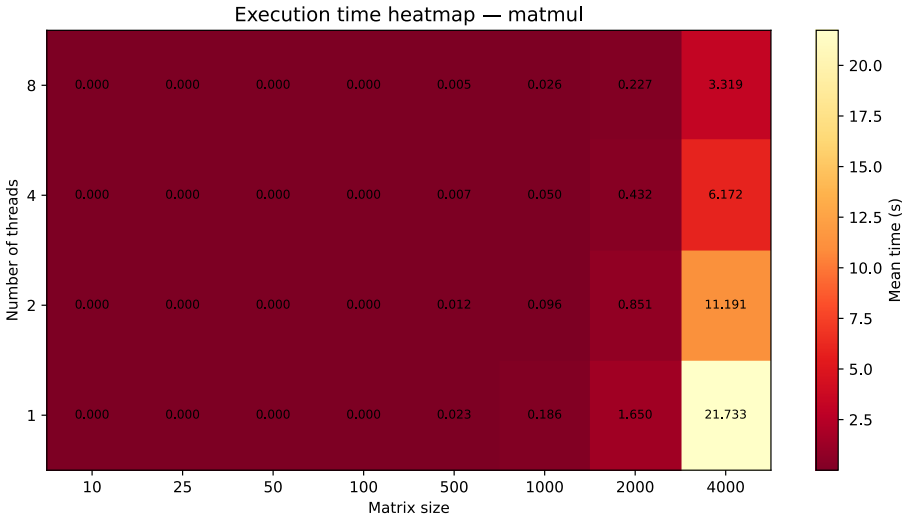
- $64 \times 64 \times 8 = 32$ KB

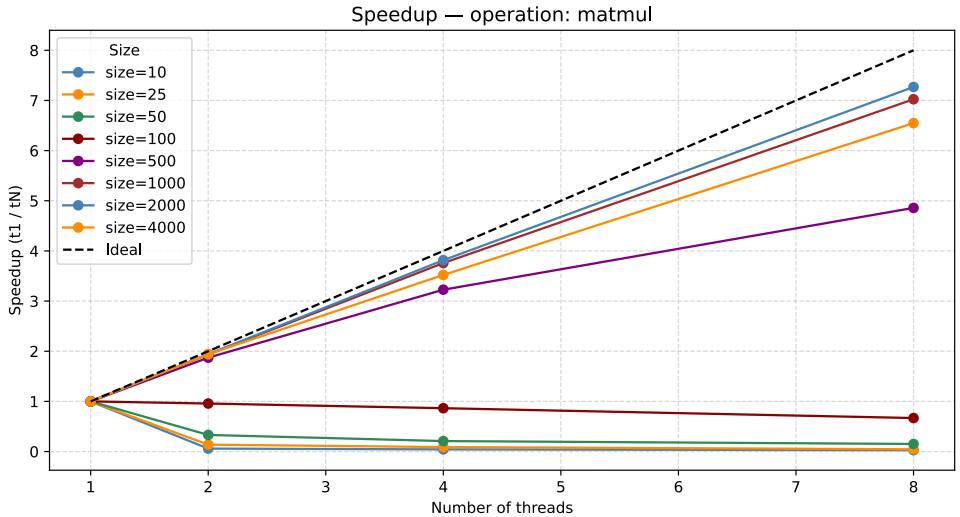
Without tiling — my PC

- For **small matrices**: thread creation time dominates
- Saturation between 4 and 8 threads (number of physical cores)
- Sub-linear speedup due to memory contention



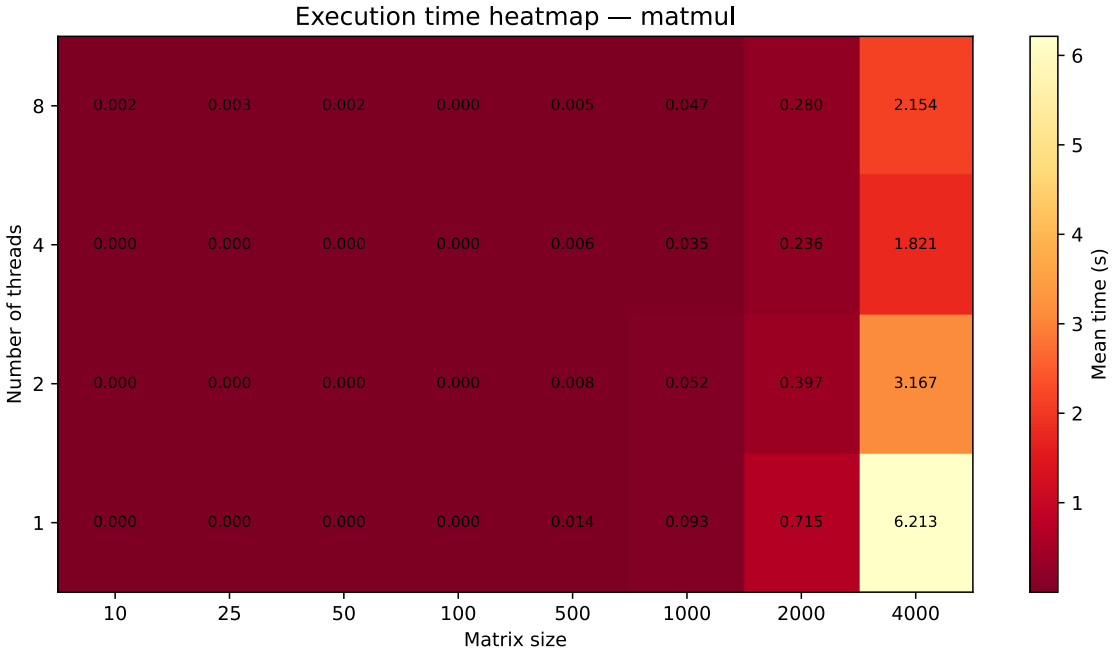
Without tiling — server



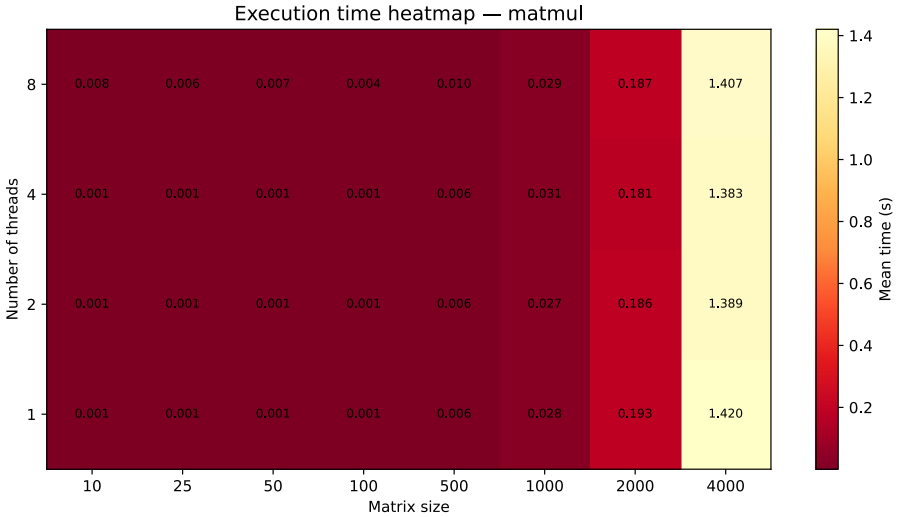


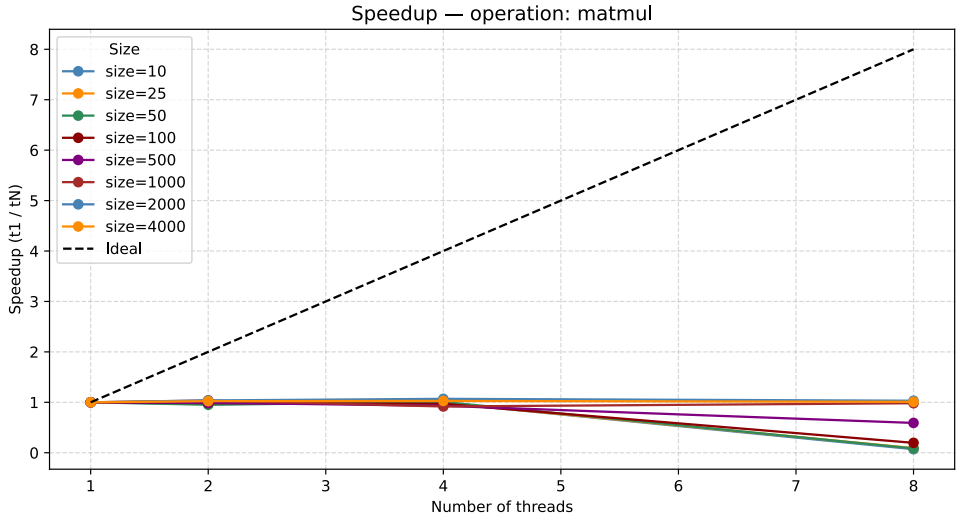
With tiling — my PC

- Improved execution time



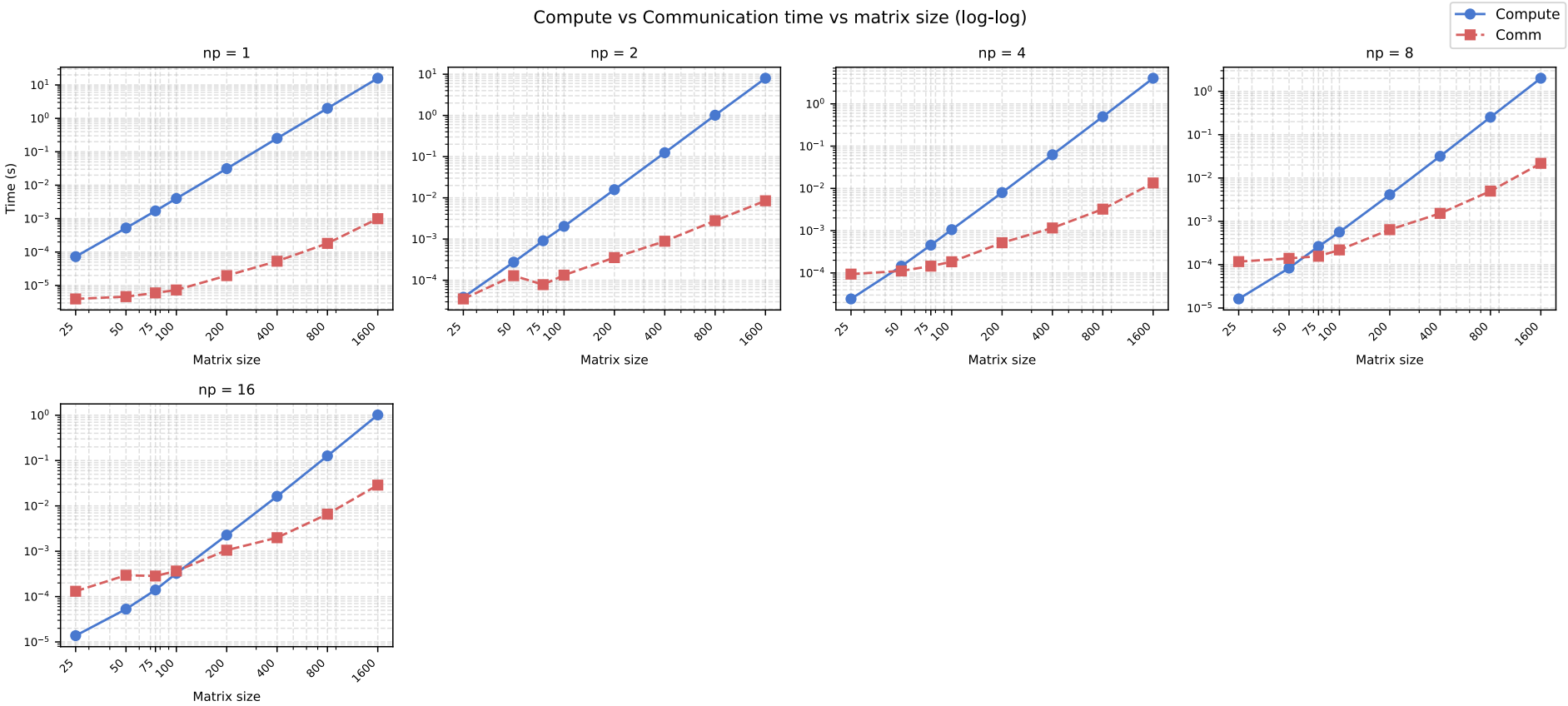
With tiling — server



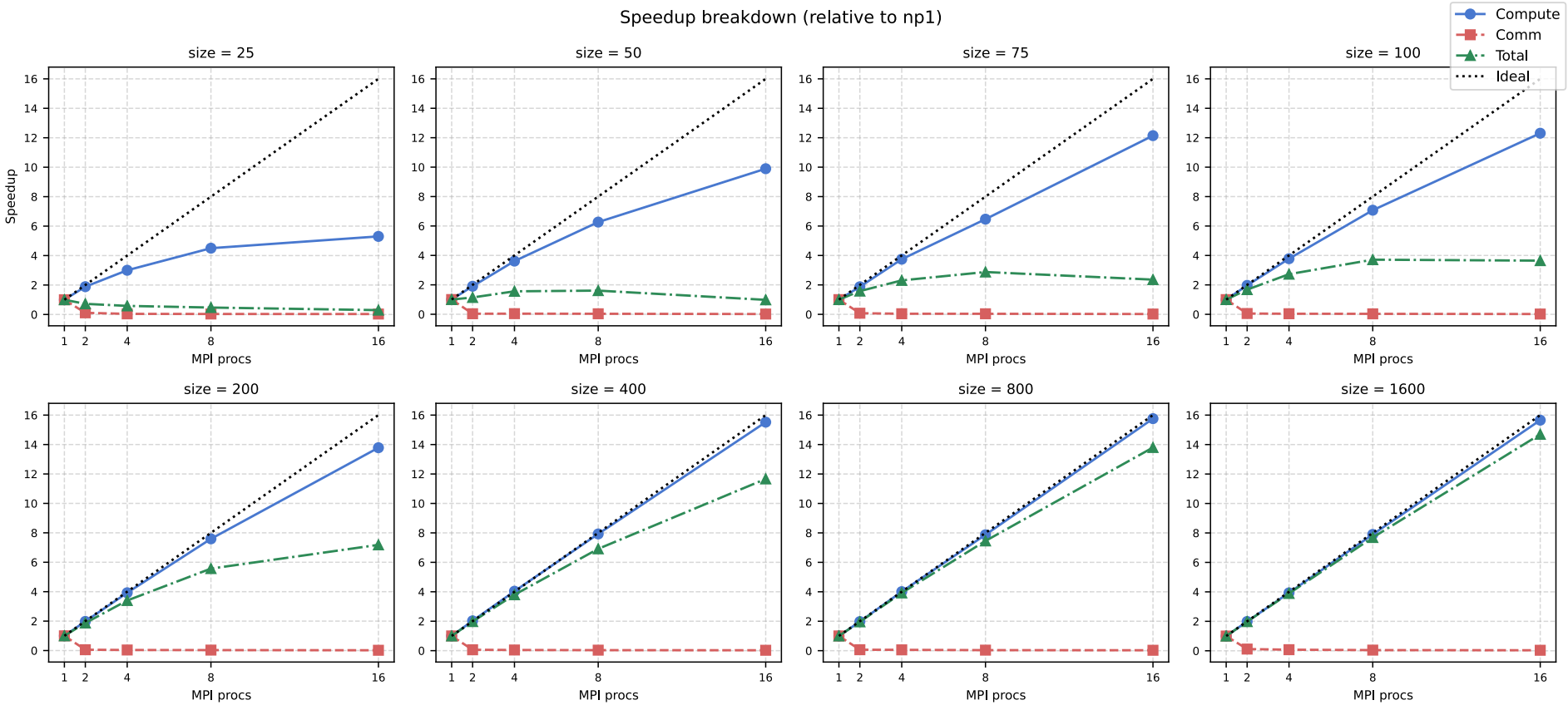


Part 3 : Distributed matrix operations (MPI)

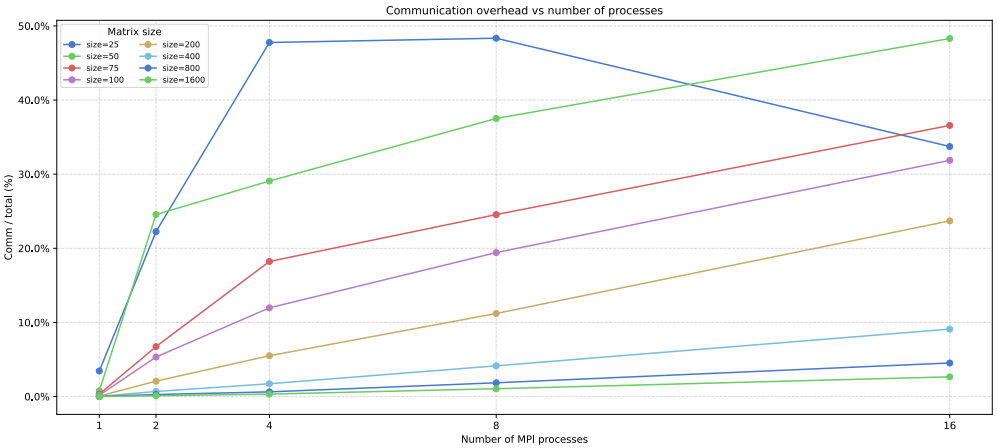
Absolute time: compute & comm vs size (log-log)



Speedup breakdown — compute / comm / total



Communication overhead



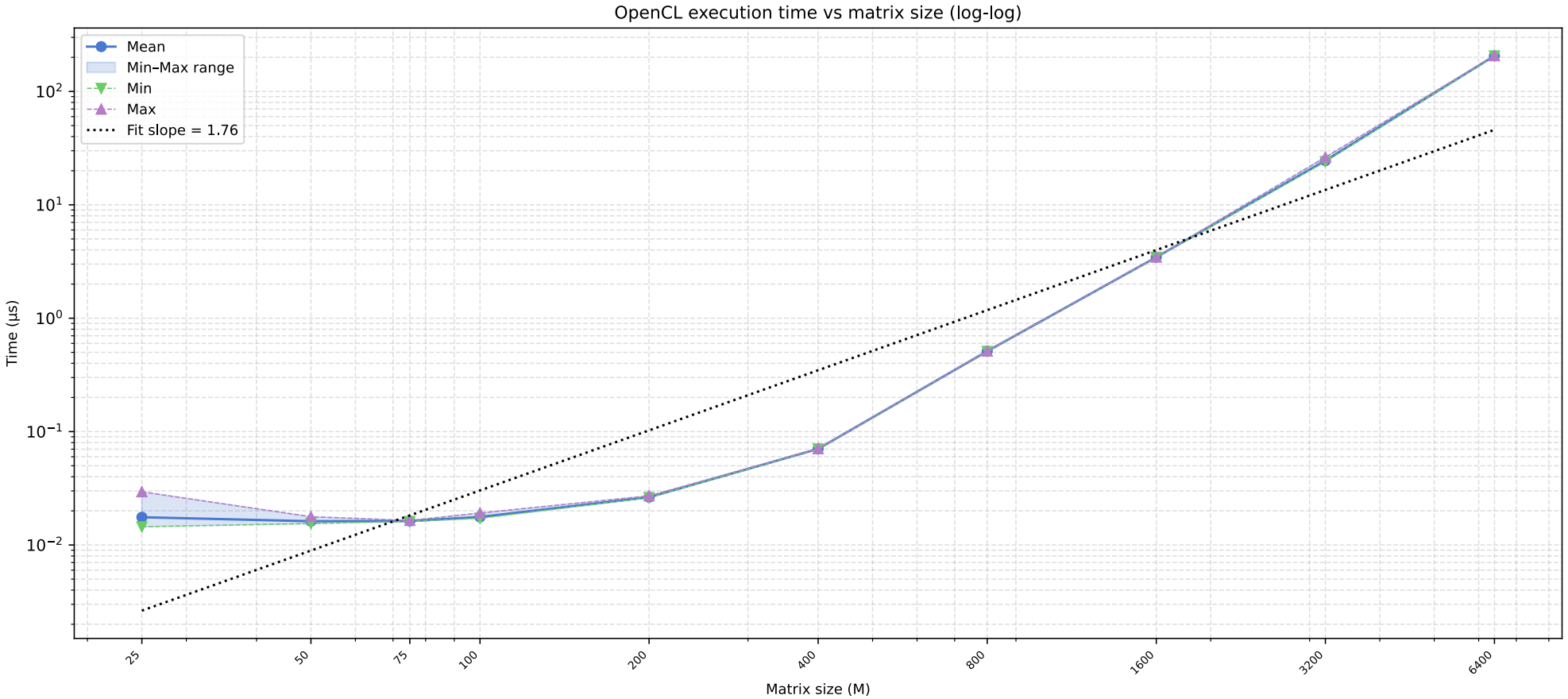
- For **small matrices**, communication dominates
- For **large matrices**, compute becomes dominant again
- Speedup close to ideal for large sizes

MPI Conclusion

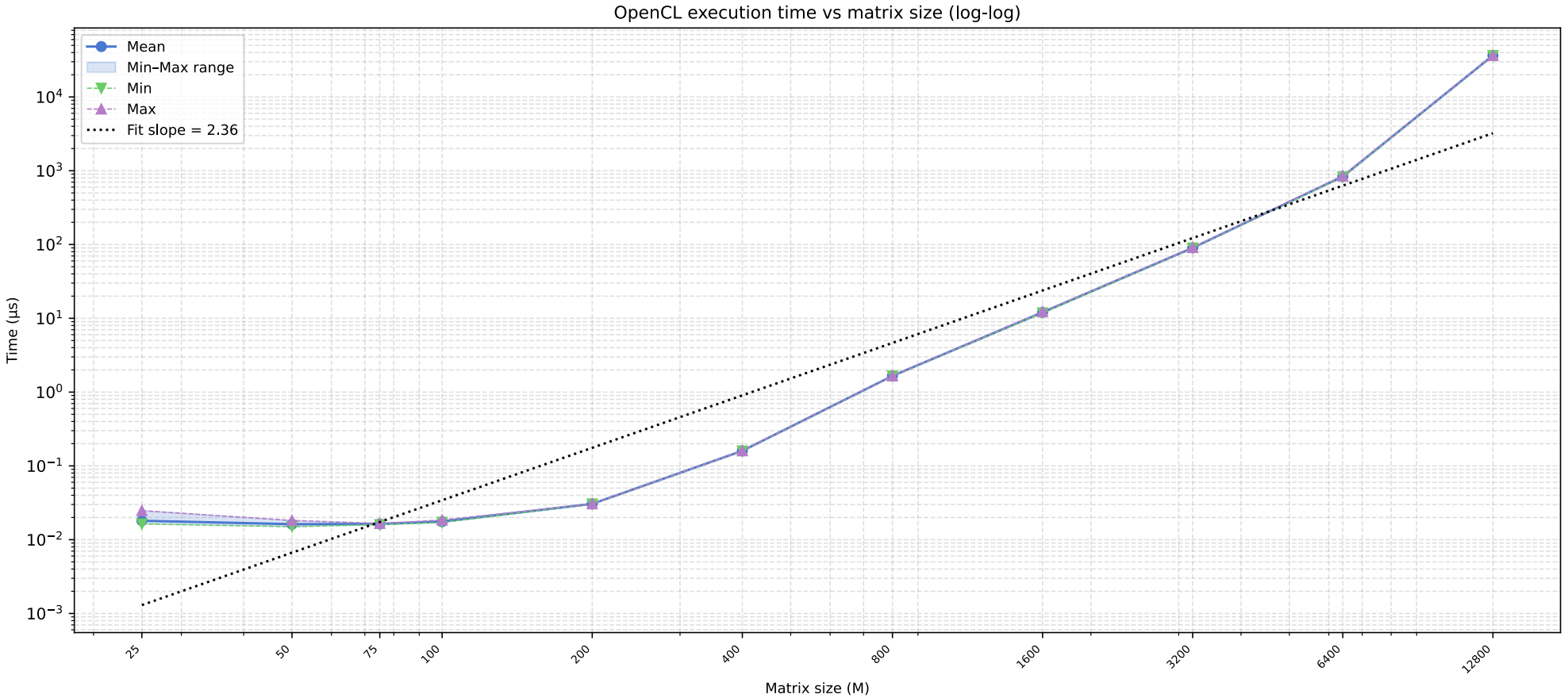
Good compute speedup, limited by Allreduce for small sizes.

Part 4 : GPU Matrix Operations (OpenCL)

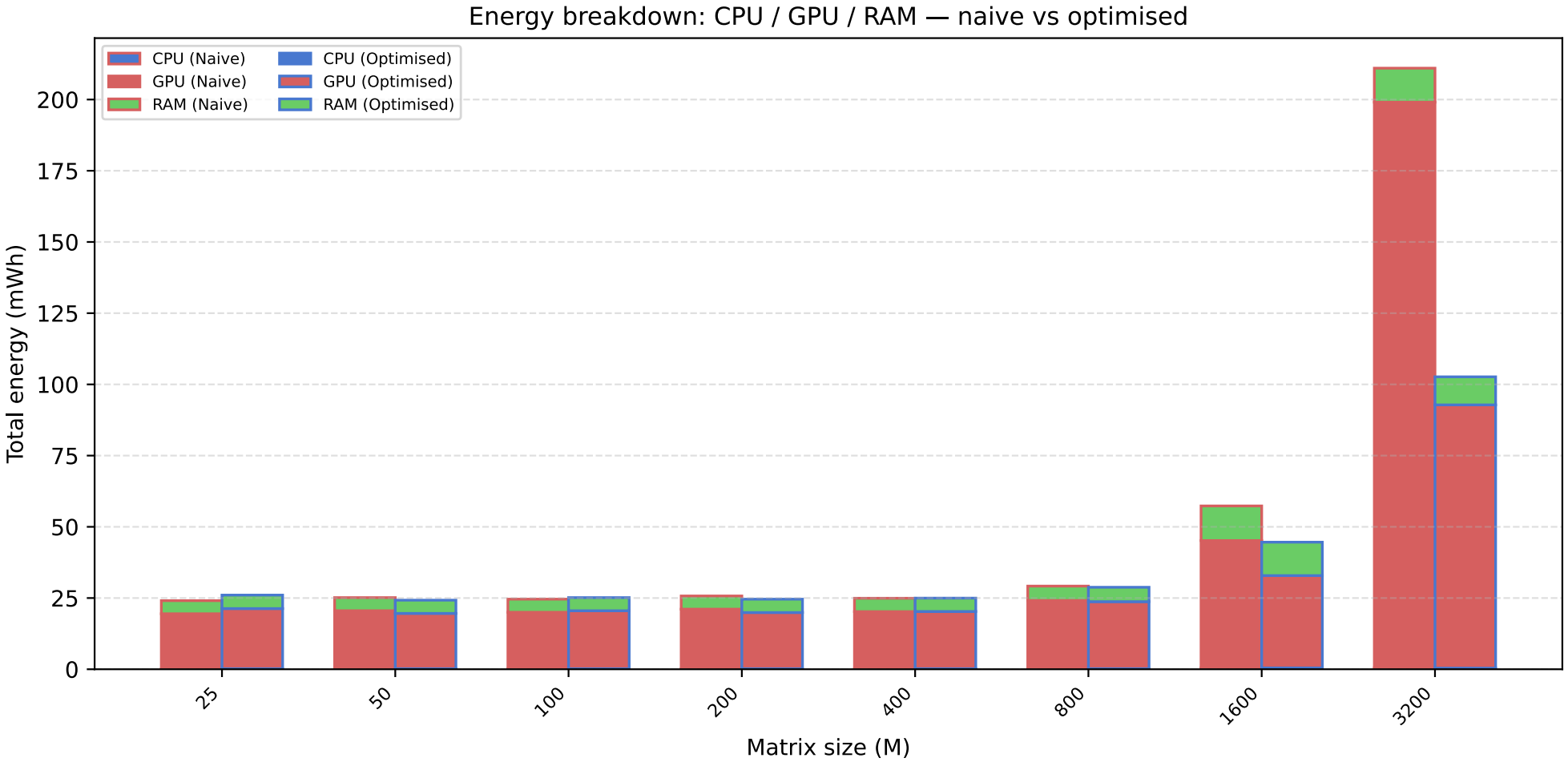
OpenCL — Result fast implementation



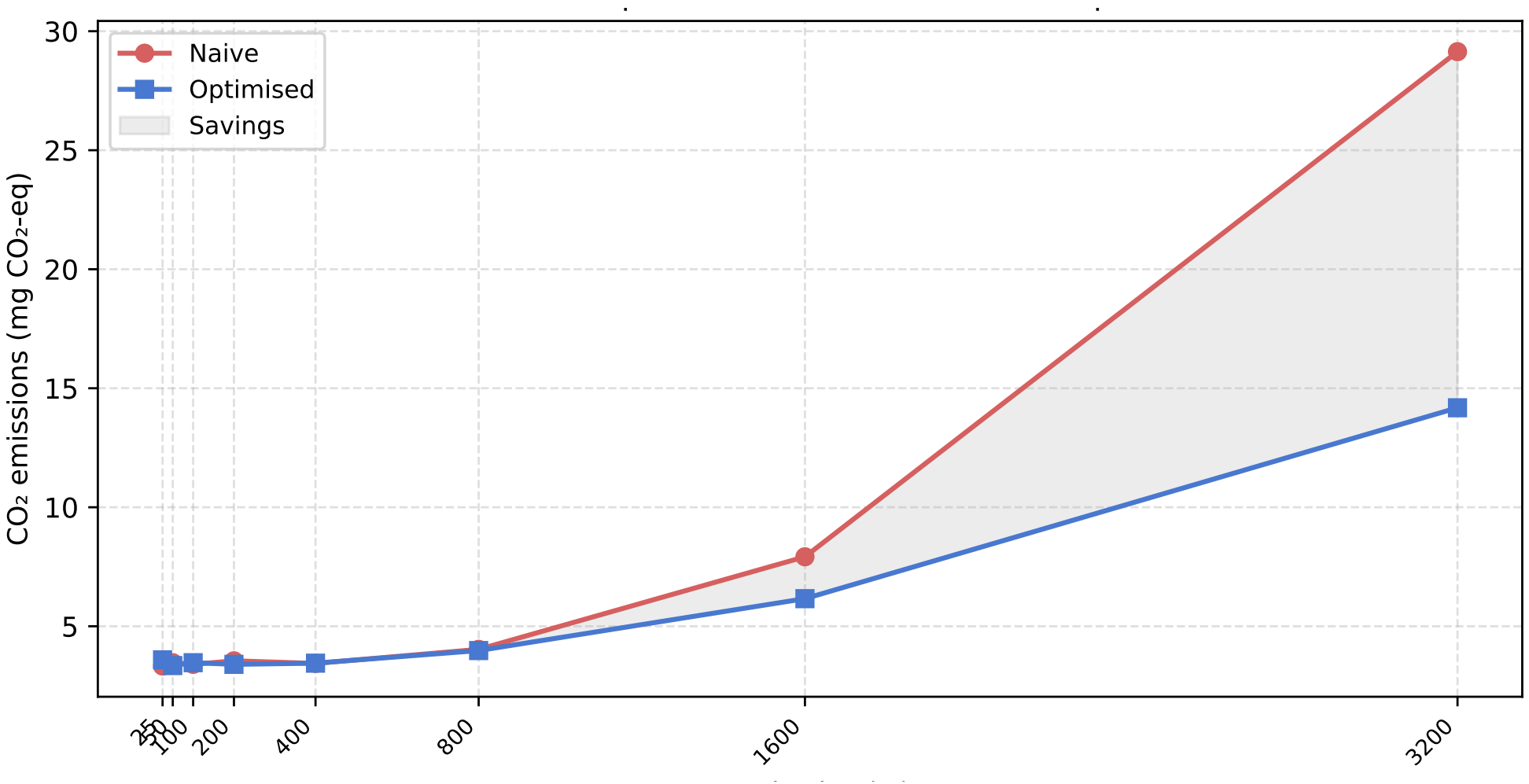
OpenCL — Result Naive implementation



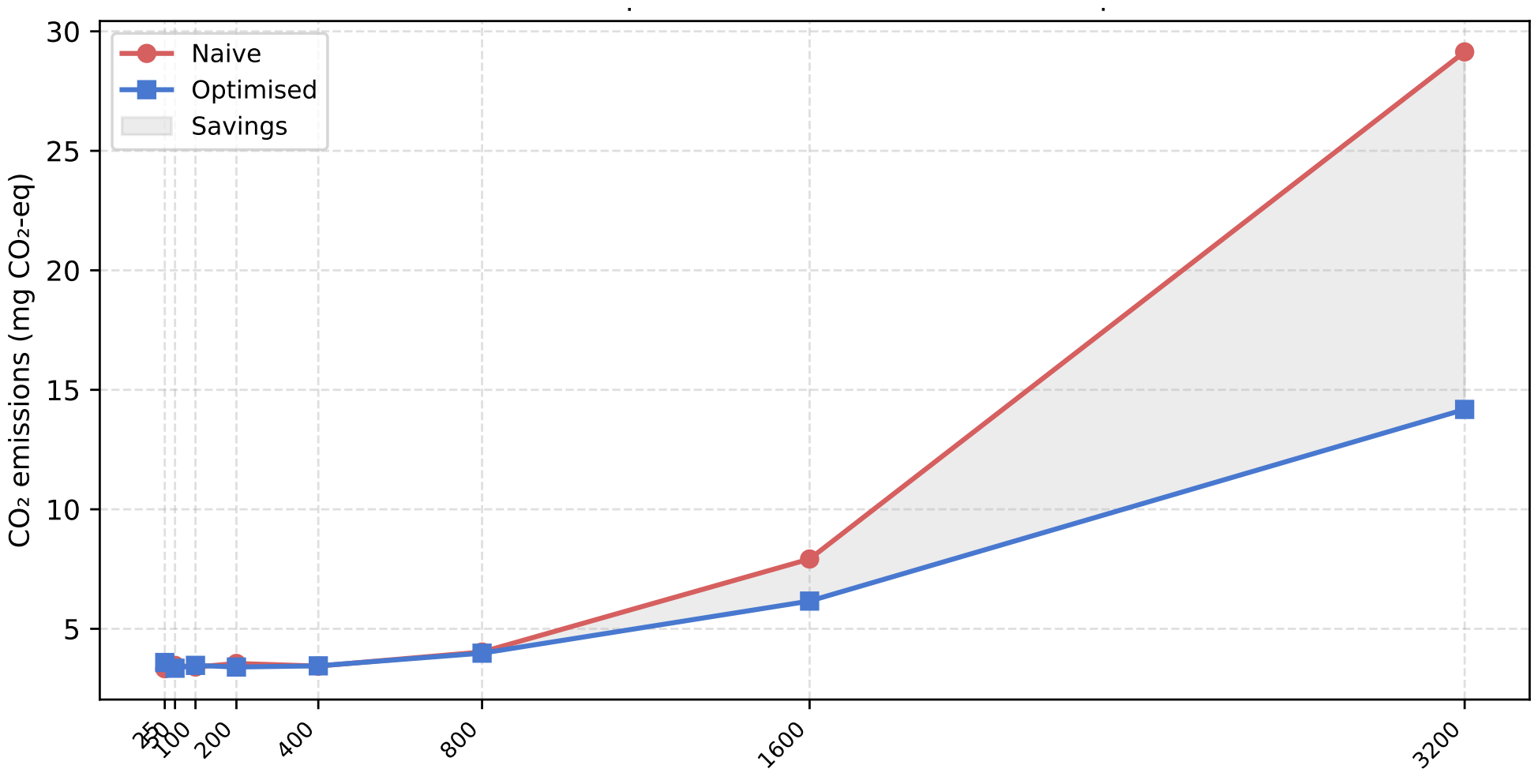
OpenCL — Energy consumption



OpenCL — Energy consumption



OpenCL — Energy consumption



OpenCL — Energy consumption